



1. Objetivos de la Unidad

Al finalizar esta unidad, el alumnado será capaz de:

- Comprender el uso de clases genéricas en colecciones.
 - Utilizar la interfaz `Collection`.
 - Trabajar con listas, conjuntos y mapas.
 - Recorrer colecciones correctamente.
 - Aplicar operaciones básicas sobre colecciones.
 - Elegir el tipo de colección adecuado según el problema.
-

2. ¿Por qué usar colecciones?

Limitaciones de los arrays:

- ✗ Tamaño fijo
- ✗ No crecen dinámicamente
- ✗ Métodos limitados
- ✗ No integran fácilmente algoritmos

Las colecciones:

- ✓ Son dinámicas
 - ✓ Incluyen muchos métodos útiles
 - ✓ Permiten trabajar con objetos
 - ✓ Integran algoritmos de ordenación y búsqueda
-

3. Jerarquía principal de colecciones

`Iterable`



`Collection`

└─ `List`

└─ Set
└─ Queue

Además:

Map (no hereda de Collection)

4. Uso de clases y métodos genéricos

Las colecciones usan genéricos para garantizar seguridad de tipos.

Ejemplo:

```
List<String> lista = new ArrayList<>();
```

Evita casting y errores en tiempo de ejecución.

5. Interfaz List

Permite elementos:

- ✓ Ordenados
 - ✓ Repetidos
 - ✓ Acceso por índice
-

5.1 ArrayList

Implementación más usada.

```
import java.util.ArrayList;  
import java.util.List;  
  
public class EjemploArrayList {  
  
    public static void main(String[] args) {
```

```
List<String> nombres = new ArrayList<>();

nombres.add("Ana");
nombres.add("Luis");
nombres.add("Carlos");

System.out.println(nombres.get(1)); // Luis

nombres.remove("Ana");

System.out.println(nombres.size());
}
```

5.2 LinkedList

Basada en lista enlazada.

Ventajas:

- Inserciones rápidas

Desventajas:

- Acceso más lento por índice

```
List<Integer> numeros = new LinkedList<>();
```

6. Interfaz Set

Características:

- ✓ No permite duplicados
 - ✓ No garantiza orden (según implementación)
-

6.1 HashSet

Más común.

```
import java.util.HashSet;
import java.util.Set;

public class EjemploHashSet {

    public static void main(String[] args) {

        Set<String> ciudades = new HashSet<>();

        ciudades.add("Madrid");
        ciudades.add("Barcelona");
        ciudades.add("Madrid"); // No se añade

        System.out.println(ciudades);
    }
}
```

6.2 TreeSet

- ✓ Ordenado automáticamente
- ✗ Requiere elementos comparables

```
Set<Integer> numeros = new TreeSet<>();
```

7. Interfaz Map

Un Map almacena pares clave–valor.

No hereda de Collection.

7.1 HashMap

```
import java.util.HashMap;
import java.util.Map;
```

```
public class EjemploHashMap {  
  
    public static void main(String[] args) {  
  
        Map<String, Integer> edades = new HashMap<>();  
  
        edades.put("Ana", 25);  
        edades.put("Luis", 30);  
  
        System.out.println(edades.get("Ana"));  
  
        for (String clave : edades.keySet()) {  
            System.out.println(clave + " -> " + edades.get(clave));  
        }  
    }  
}
```

7.2 TreeMap

✓ Ordenado por clave

```
Map<String, Integer> mapaOrdenado = new TreeMap<>();
```

8. Recorrido de colecciones

8.1 For-each

```
for (String nombre : nombres) {  
    System.out.println(nombre);  
}
```

8.2 Iterator

```
import java.util.Iterator;
```

```
Iterator<String> it = nombres.iterator();

while (it.hasNext()) {
    System.out.println(it.next());
}
```

Necesario si se eliminan elementos durante el recorrido:

```
Iterator<String> it = nombres.iterator();

while (it.hasNext()) {
    if (it.next().equals("Luis")) {
        it.remove();
    }
}
```

9. Operaciones comunes

Método	Descripción
add()	Añadir elemento
remove()	Eliminar
contains()	Verificar existencia
size()	Tamaño
clear()	Vaciar
isEmpty()	Verificar si está vacía

10. Ordenación con Collections

```
import java.util.Collections;

Collections.sort(nombres);
```

11. Ejemplo integrador completo

```
import java.util.*;

class Producto {

    private String nombre;
    private double precio;

    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    public String getNombre() {
        return nombre;
    }

    public double getPrecio() {
        return precio;
    }

    @Override
    public String toString() {
        return nombre + " - " + precio;
    }
}

public class Main {

    public static void main(String[] args) {

        List<Producto> productos = new ArrayList<>();

        productos.add(new Producto("Teclado", 30));
        productos.add(new Producto("Ratón", 15));
        productos.add(new Producto("Monitor", 200));

        for (Producto p : productos) {
            System.out.println(p);
        }
    }
}
```

```
}  
}
```

12. Errores comunes del alumnado

- ✗ No usar genéricos
 - ✗ Intentar usar índices en Set
 - ✗ No entender diferencia List vs Set
 - ✗ No sobrescribir equals() en objetos usados en HashSet
 - ✗ No entender clave-valor en Map
-

13. Actividades propuestas

- 1 Crear una lista de números y ordenarla.
 - 2 Crear un Set que elimine duplicados.
 - 3 Crear un Map de productos y precios.
 - 4 Implementar clase Persona con Comparable.
-

14. Resumen de la Unidad

En esta unidad hemos aprendido:

- Qué son las colecciones.
- List, Set y Map.
- ArrayList, HashSet y HashMap.
- Recorridos.
- Ordenación.
- Comparable vs Comparator.
- Uso de genéricos.

15. ¿Cuál usar?

Veamos cuál usar depende de lo que necesitemos:

List

Una **List** es una colección ordenada que permite elementos duplicados.

Características:

- Mantiene el orden de inserción.
- Permite elementos repetidos.
- Permite acceder a los elementos por índice.

Ejemplos de implementación:

- **ArrayList:**
Usa un array dinámico internamente. Es rápida para acceder a elementos (**get**), pero lenta para insertar o eliminar en posiciones intermedias.
- **LinkedList:**
Usa una lista enlazada. Es más eficiente al insertar o eliminar elementos, pero más lenta al acceder por índice.

Set

Un **Set** es una colección que NO permite elementos duplicados.

Características:

- No permite elementos repetidos.
- No tiene acceso por índice.
- El orden depende de la implementación.

Ejemplos de implementación:

- **HashSet:**
No mantiene ningún orden. Es la opción más rápida en general.
- **LinkedHashSet:**
Mantiene el orden de inserción de los elementos.
- **TreeSet:**
Ordena automáticamente los elementos (orden natural o mediante un comparador).

Map

Un **Map** almacena datos en pares clave-valor (key-value).

Características:

- Las claves son únicas.

- Los valores pueden repetirse.
- Permite acceder a los valores mediante la clave.
- No extiende de **Collection**.

Ejemplos de implementación:

- **HashMap:**
No mantiene orden. Es el más usado por su rapidez.
- **LinkedHashMap:**
Mantiene el orden de inserción.
- **TreeMap:**
Ordena automáticamente las claves.

Característica	List	Set	Map
Orden	Sí	Depende	Depende
Elementos duplicados	Sí	No	Claves no, valores sí
Acceso por índice	Sí	No	No
Estructura	Elementos	Elementos únicos	Clave - Valor

BLOQUE 1 — ArrayList

1. Crear `ArrayList<String>`, añadir 10 nombres pidiéndolos por pantalla y mostrarlos.
2. Crear `ArrayList<Integer>`, añádele 5 valores random y muestra solo los pares
3. Muestra el tamaño de la lista, el primer elemento y el último. Además muestra el mayor

PISTA: Usa un método de Collections

4. Elimina un elemento concreto de la lista utilizando su valor. Sólo el primero. (En caso de que haya más de uno)
5. Elimina todos los elementos que contengan la letra "a".
6. Convierte un `ArrayList<String>` en un array.

BLOQUE 2 — Recorridos e Iterator

Métodos clave:

`hasNext()` → ¿hay más elementos?

`next()` → dame el siguiente

`remove()` → elimina el actual

7. Recorre un `ArrayList<String>` utilizando un `Iterator` y muestra todos sus elementos por pantalla.
8. Recorre un `ArrayList<Integer>` utilizando un `Iterator` y calcula la suma total de sus elementos.
9. Recorre un `ArrayList<Integer>` utilizando un `Iterator` y elimina todos los números pares de la lista.

Métodos clave de `ListIterator`:

`hasPrevious()` → devuelve true si hay un elemento antes del actual

`previous()` → devuelve el elemento anterior en la lista

`nextIndex()` → devuelve el índice del elemento que sería devuelto por `next()`

`previousIndex()` → devuelve el índice del elemento que sería devuelto por `previous()`

`set()` → reemplaza el último elemento devuelto por `next()` o `previous()`

`add()` → inserta un elemento en la posición actual

`remove()` → elimina el último elemento devuelto por `next()` o `previous()`

10. Recorre un `ArrayList<Integer>` utilizando un `ListIterator`, elimina los números pares y sustituye los impares por su doble.

11. Recorre un `ArrayList<Integer>` usando un `ListIterator` y realiza lo siguiente:

- Cada vez que encuentres un número múltiplo de 5, **inserta justo después un 0** usando `add()`.
- Después, imprime la lista final recorriéndola de atrás hacia adelante.

BLOQUE 3 — LinkedList

Métodos clave de LinkedList:

`add()` → añade un elemento al final de la lista
`addFirst()` → añade un elemento al inicio
`addLast()` → añade un elemento al final
`add(index, element)` → inserta un elemento en una posición específica
`get()` → obtiene un elemento por índice
`getFirst()` → obtiene el primer elemento
`getLast()` → obtiene el último elemento
`set()` → reemplaza un elemento en una posición concreta
`remove()` → elimina un elemento por valor
`remove(index)` → elimina un elemento por índice
`removeFirst()` → elimina el primer elemento
`removeLast()` → elimina el último elemento
`contains()` → comprueba si existe un elemento
`size()` → devuelve el número de elementos
`clear()` → elimina todos los elementos

Métodos tipo cola (Queue):

`offer()` → añade un elemento al final
`offerFirst()` → añade al inicio
`offerLast()` → añade al final
`peek()` → obtiene el primer elemento sin eliminarlo
`poll()` → obtiene y elimina el primer elemento
`pollFirst()` → elimina el primer elemento
`pollLast()` → elimina el último elemento

Métodos tipo pila (Stack):

`push()` → añade un elemento al inicio (como pila)
`pop()` → elimina y devuelve el primer elemento

Otros útiles:

`indexOf()` → devuelve el índice de un elemento
`lastIndexOf()` → devuelve la última posición de un elemento

toArray() → convierte la lista en un array
iterator() → devuelve un iterador
descendingIterator() → iterador en orden inverso

Ejercicios

12. Crea una `LinkedList<Integer>` y añade elementos al inicio y al final.
13. Simula una cola FIFO (Primero en Entrar, Primero en Salir) usando `LinkedList`:
añade elementos y luego procésalos eliminándolos.
14. Dado un array de enteros, insértalos en una `LinkedList`:
 - Si el número es mayor o igual que el anterior, añádelo al inicio.
 - Si es menor, añádelo al final.
15. Recorre una `LinkedList<Integer>` y elimina todos los números pares usando un `Iterator`.

BLOQUE 4 — Set

Métodos clave de Set:

add() → añade un elemento si no está ya presente
remove() → elimina un elemento del conjunto
contains() → comprueba si un elemento existe en el conjunto
size() → devuelve el número de elementos
isEmpty() → comprueba si el conjunto está vacío
clear() → elimina todos los elementos

Métodos de comparación y operaciones:

equals() → compara si dos conjuntos son iguales
addAll() → añade todos los elementos de otra colección
removeAll() → elimina los elementos que estén en otra colección
retainAll() → conserva solo los elementos comunes con otra colección
containsAll() → comprueba si contiene todos los elementos de otra colección

Métodos de iteración:

iterator() → devuelve un iterador para recorrer el conjunto
forEach() → recorre los elementos aplicando una acción

Otros útiles:

toArray() → convierte el conjunto en un array
hashCode() → devuelve el código hash del conjunto

Ejercicios

16. Crea un `HashSet<String>` e introduce valores duplicados. Comprueba que no se almacenan repetidos.

17. Dado un `ArrayList<String>`, elimina los duplicados utilizando un `Set`.

18. Dado un `Set<Integer>` con números 3 aleatorios del 1 al 100, comprueba si existe un número que sea múltiplo de 3 y mayor que 50. Si existe, muéstralo; si no, indica que no hay ninguno.

19. Dado un `Set<Integer>`, crea otro Set con solo los elementos mayores que 10 utilizando un bucle.

BLOQUE 5 — Map

Métodos clave de Map:

`put()` → añade o reemplaza un par clave-valor
`get()` → obtiene el valor asociado a una clave
`remove()` → elimina un elemento por clave
`containsKey()` → comprueba si existe una clave
`containsValue()` → comprueba si existe un valor
`size()` → devuelve el número de elementos
`isEmpty()` → comprueba si el mapa está vacío
`clear()` → elimina todos los elementos

Métodos de acceso a colecciones:

`keySet()` → devuelve un conjunto con todas las claves
`values()` → devuelve una colección con todos los valores
`entrySet()` → devuelve un conjunto de pares clave-valor

Métodos útiles:

`putIfAbsent()` → añade un valor solo si la clave no existe
`replace()` → reemplaza el valor de una clave
`getOrDefault()` → obtiene un valor o devuelve uno por defecto si no existe
`forEach()` → recorre todos los pares clave-valor

Ejercicios

20. Crea un `HashMap<String, Integer>` para almacenar nombres y edades.

21. Muestra la edad de una persona a partir de su nombre.
Recorre un `Map` utilizando `keySet()` y un `Iterator`.

22. Dado un `Map<String, Integer>`, muestra solo las entradas cuyo valor sea mayor de 18 utilizando `entrySet()`.

23. Crea un programa que:

1. Pida nombres al usuario y los guarde en un `ArrayList`.
2. Elimine duplicados usando un `Set`.
3. Cree un `Map<String, Integer>` donde la clave sea el nombre y el valor la longitud del nombre.
4. Recorra el mapa y muestre todos los resultados.